FFPA: Efficient Flash Prefill Attention for Large Head Dimensions via Split-D

Technical Report

DefTruth[†] Butterfingrz[‡]

[†]vipshop.com [‡]Shanghai University

June 2026

Abstract

Attention mechanisms with large head dimensions ($D > 256$) are increasingly common in diffusion transformers and video generation models, yet standard FlashAttention kernels—optimized for $D \leq 256$—suffer from shared memory exhaustion and register pressure when D grows beyond 256. We present FFPA (Faster Flash Prefill Attention), which introduces **Split-D**, a head-dimension chunking strategy that decomposes $QK^\top$ and PV into sub-operations over D-slices. Split-D reduces SRAM complexity from $O(B_c \times D)$ to $O(B_c \times D_{\text{chunk}}) \approx O(1)$, keeping active register pressure bounded at $O(D_{\text{chunk}})$. The forward pass uses Split-D universally with no precision issues, suitable for both training and inference on any GPU. For the backward pass, two hardware-driven strategies are provided: an fp32 gradient buffer for compute-limited SM≤89 GPUs, and a Hopper-specialized CuTe DSL kernel with full-D WGMMA forward and recompute-based backward for SM≥90 GPUs. Across H200, H800, H20, RTX 5090, and L20, FFPA achieves **1.5–6.1**× speedup over PyTorch SDPA for $D \in [320, 1024]$, reaching **426 TFLOPS** on H200, and **1.4–1.5**× end-to-end training throughput on Gemma4-31B. Code is available at `https://github.com/xlite-dev/ffpa-attn`.

1 Introduction

The attention mechanism [1] is the computational heart of modern transformer models. While most large language models use moderate head dimensions ($D = 64$ or 128), a growing class of architectures—including diffusion transformers (DiTs) [5], video diffusion models [6], and certain vision transformers—employ *large* head dimensions of $D = 320$, 512, or even 1024. In these regimes, attention accounts for a dominant fraction of end-to-end inference and training time.

FlashAttention [2–4] revolutionized attention computation by fusing the $QK^\top$–softmax–PV pipeline into a single GPU kernel operating on tiles in SRAM, avoiding $O(N^2)$ writes to high-bandwidth memory (HBM). The core tiling strategy partitions the sequence dimension N into blocks of size B_r (query rows) and B_c (key/value rows), while keeping the *full* head dimension D as the inner dimension of each matrix multiply. This design is optimal when D is small enough that intermediate score tiles and output accumulators fit in shared memory and the register file. However, when $D > 256$, the shared memory footprint of a full-D key/value tile grows to $O(B_c \times D)$, exceeding the SRAM budget and forcing repeated global memory accesses. Simultaneously, the register footprint of the output accumulator ($B_r \times D$) exceeds the GPU register budget, causing spilling to local memory.

We observe that the large-D bottleneck can be addressed by applying tiling *recursively* to the head dimension itself. Our key insight is that the online-softmax recurrence is *independent* of how the $QK^\top$ matmul is computed internally: as long as the final $[B_r, B_c]$ score tile is correctly accumulated, the softmax statistics m_i and ℓ_i remain valid regardless of whether the dot product was computed in one full-D operation or accumulated over multiple D-chunks. This leads to **Split-D**, a head-dimension chunking strategy that we integrate into a complete FlashAttention-style tiled kernel.

A critical challenge arising from Split-D is numerical precision in the **backward pass**. When gradients are accumulated across multiple D-chunks, the bf16 load-modify-store round-trip introduces precision loss proportional to the number of contributing tiles. The forward pass is unaffected—online softmax with fp32 accumulation keeps accuracy within standard mixed-precision bounds, making FFPA equally suitable for inference on any GPU. For the backward pass, we address precision with a **hardware-driven strategy**: on compute-limited SM≤89 GPUs

(e.g., L20 at 119.5 dense FP16 TFLOPS), we use an fp32 gradient buffer that trades IO bandwidth for full-precision accumulation; on compute-rich SM≥90 GPUs (e.g., H200 at ∼989 dense FP16 TFLOPS), we adopt a recompute-based approach that re-materializes the softmax and its gradient across D-chunks, exploiting surplus tensor core throughput to eliminate bf16 round-trip errors.

We make the following contributions:

1. **Split-D tiling**: A head-dimension chunking strategy that *solves* the shared memory bottleneck ($O(1)$ SRAM complexity with respect to D) and *alleviates* register pressure by batching operations over D_{chunk}-sized slices.
2. **Generic forward/backward kernels** (CUDA & Triton, SM≥80) with a hardware-driven precision strategy: fp32 gradient buffer for SM≤89, where IO overhead is cheaper than recompute.
3. **Hopper-specialized kernels** (CuTe DSL, SM≥90, $D = 512$): full-D $QK^\top$ via a single WGMMA instruction in the forward pass, and a recompute-based backward with fp32 precision channels that eliminate bf16 round-trip loss.
4. **Multi-backend system** spanning CUDA, Triton, and CuTe DSL with automatic dispatch based on GPU architecture and head dimension.
5. **Comprehensive evaluation** across many GPU architectures (H200, H800, H20, RTX 5090, L20) and an end-to-end training verification on Gemma4-31B, achieving 1.5×–6.1× micro-benchmark speedup and 1.4–1.5× training throughput improvement.

2 Background

2.1 Multi-Head Attention

Given query, key, and value matrices $Q, K, V \in \mathbb{R}^{N\times D}$ for a single attention head, the standard scaled dot-product attention computes:

$$S = \alpha\, QK^\top, \quad \alpha = 1/\sqrt{D}, \tag{1}$$

$$P = \mathrm{softmax}(S), \tag{2}$$

$$O = PV, \tag{3}$$

where softmax is applied row-wise. In practice, a causal mask or additive bias may be applied to S before the softmax. The backward pass computes gradients dQ, dK, dV from the output gradient dO:

$$dP = dO\, V^\top - \mathrm{diag}(\mathrm{rowsum}(dO \odot O))\, P, \tag{4}$$

$$dS = P \odot dP, \tag{5}$$

$$dQ = \alpha\, dS\, K, \quad dK = \alpha\, dS^\top Q, \quad dV = P^\top dO. \tag{6}$$

2.2 FlashAttention and Online Softmax

FlashAttention [2] avoids materializing the $N \times N$ attention matrix in HBM by tiling over the sequence dimension. For a query block of B_r rows, the algorithm streams $T_c = \lceil N/B_c \rceil$ key/value blocks through SRAM, maintaining running statistics $m_i, \ell_i \in \mathbb{R}^{B_r}$ that track the row-wise max and sum of $\exp(S_{i(j)})$ across all KV blocks $j = 0, \ldots, T_c - 1$:

$$m_i^{(j+1)} = \max(m_i^{(j)}, \mathrm{rowmax}(S_{i(j)})), \tag{7}$$

$$\ell_i^{(j+1)} = \ell_i^{(j)} \exp(m_i^{(j)} - m_i^{(j+1)}) + \mathrm{rowsum}\big(\exp(S_{i(j)} - m_i^{(j+1)})\big). \tag{8}$$

The output accumulator is rescaled at each step: $O_i^{(j+1)} = \mathrm{diag}\big(\exp(m_i^{(j)} - m_i^{(j+1)})\big)\, O_i^{(j)} + P_{i(j)} V_j$.

FlashAttention-2 [3] improved work partitioning by assigning each program a query block and optimizing the loop order to reduce shared memory accesses. FlashAttention-3 [4] introduced warp-specialized producer-consumer pipelining and FP8 support for Hopper GPUs.

2.3 The Large Head-Dimension Bottleneck

Standard FlashAttention treats D as a *fused* dimension in every matrix multiply. This creates two bottlenecks when D is large:

Shared memory (SMEM). Each KV block requires loading a full-D key tile of size $B_c \times D$ and a full-D value tile of the same size. At $D = 512$ and $B_c = 64$ with fp16 storage, this demands 64 KB per tile, and with double-buffering the SMEM footprint can exceed the typical 128–228 KB budget on modern GPUs. The standard workaround—reducing B_c—increases the number of HBM round-trips, negating the original purpose of tiling.

Register file. The output accumulator $O_i \in \mathbb{R}^{B_r \times D}$ alone occupies $B_r \times D$ registers. With $B_r = 128$ and $D = 512$, this is 65,536 float values (256 KB), far exceeding the register file of any current GPU (even Hopper's 256 KB is shared across all warps in an SM). The compiler spills these tensors to local memory, causing significant latency.

Split-D addresses the **SMEM bottleneck** directly by reducing the per-tile SRAM requirement to $O(B_c \times D_{\text{chunk}})$, and **alleviates** register pressure by ensuring only D_{chunk}-sized slices of Q, K, and V are live in registers at any time.

3 FFPA: Algorithm

FFPA provides a two-tier algorithm family whose choice of backward precision strategy is determined by hardware compute capability. Section 3.1 establishes the Split-D principle. Section 3.2 describes the generic algorithm for SM≥80 GPUs (CUDA and Triton backends). Section 3.3 presents the Hopper-specialized CuTe DSL variant.

3.1 Split-D: Principle and Correctness

Split-D decomposes each D-dimensional matrix multiply into a sequence of smaller multiplies over head-dimension chunks of size D_{qk} (for $QK^\top$) and D_v (for PV).

SMEM analysis. In standard FlashAttention, loading a key tile requires $B_c \times D$ elements in shared memory. With Split-D, only a single D_{qk}-sized chunk of K ($B_c \times D_{\text{qk}}$ elements) and a single D_v-sized chunk of V ($B_c \times D_v$ elements) reside in SMEM at any time. The SRAM complexity drops from $O(B_c \times D)$ to $O(B_c \times \max(D_{\text{qk}}, D_v))$. With typical values $D_{\text{qk}} = D_v = 64$, this is a constant with respect to D: approximately $O(1)$.

Register analysis. The output accumulator still requires $O(B_r \times D)$ total storage across all V-groups. However, only one V-group accumulator and one Q/K chunk are actively used in computation at any moment, keeping the *active* register pressure at $O(B_r \times D_v)$. The total register complexity remains $O(D/4)$ (one register per 4 elements when packed), but the active set is bounded by D_{chunk}, enabling the compiler to schedule spills more efficiently.

Correctness. The online softmax recurrence (Eqs. 7–8) depends only on the final $[B_r, B_c]$ score tile $S_{i(j)}$, not on how many partial dot products were summed to produce it. As long as $S_{i(j)} = \sum_c Q_i^{(c)} K_j^{(c)\top}$ is correctly accumulated across all D_{qk}-chunks before the softmax step, the statistics m_i and ℓ_i remain identical to the full-D computation. The same reasoning applies to the PV phase: the rescaling factor α_i and softmax probabilities $P_{i(j)}$ are computed once per KV block and reused for all V-groups.

3.2 Generic Split-D Algorithm (CUDA & Triton, SM≥80)

The generic Split-D algorithm targets a broad range of GPU architectures (SM≥80: Ampere, Ada, Hopper, Blackwell) through CUDA C++ and Triton implementations. Both forward and backward passes apply full Split-D tiling.

3.2.1 Forward Pass

Algorithm 1 presents the Split-D forward pass for a single program that owns one B_r-sized query block. The outer loop iterates over KV blocks; inside each KV block, a **QK accumulation loop** sums partial dot products over $N_{\text{qk}} = \lceil D/D_{\text{qk}} \rceil$ chunks, followed by online softmax and a **PV accumulation loop** over $N_v = \lceil D/D_v \rceil$ V-groups.

3.2.2 Backward Pass and Precision Strategy

The backward pass computes dQ, dK, and dV from dO and the saved LSE. FFPA uses two specialized kernels:

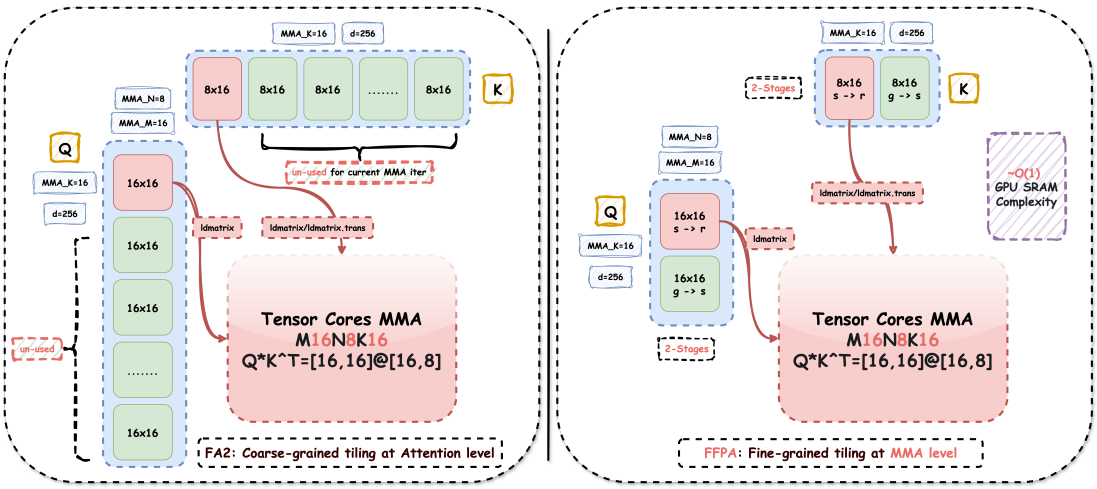


Figure 1: Split-D tiling strategy. The head dimension D is chunked into D_{qk}-sized slices for $QK^\top$ accumulation and D_v-sized groups for PV computation. Only one chunk resides in SMEM at any time.

dKdV Kernel. Iterates over KV blocks as the outer loop. For each KV block j, it streams all query blocks through SRAM, recomputing $S_{i(j)}$, $P_{i(j)}$, and the softmax statistics via Split-D. It accumulates dV_j and dK_j to global memory using a load-modify-store pattern.

dQ Kernel. Iterates over Q-blocks as the outer loop, recomputing $S_{i(j)}$ for all KV blocks and accumulating dQ_i. A pre-processing kernel computes $\Delta_i = \text{rowsum}(dO_i \odot O_i)$, enabling $dS_{i(j)} = P_{i(j)} \odot (dO_i V_j^\top - \Delta_i)$.

Algorithm 2 outlines the Split-D dKdV backward pass; the dQ kernel follows a symmetric structure with Q-blocks as the outer loop.

Precision challenge. When gradients are stored in bf16 (7-bit mantissa), the load-modify-store pattern in dKdV and dQ introduces a bf16 round-trip at every partial update. With up to $T_c = \lceil N/B_c \rceil = 128$ contributing tiles per element, the accumulated error can exceed 10^{-2} in causal backward, where contributions are unequally weighted near the diagonal.

FFPA solution: fp32 gradient buffer. FFPA provides the option to allocate DK, DV, and DQ tensors in fp32 (`grad_kv_storage_dtype = torch.float32`, `grad_q_storage_dtype = torch.float32`). This doubles the IO bandwidth for gradient reads and writes (fp32 is 2× bf16), but eliminates bf16 round-trip errors entirely. On compute-limited SM≤89 GPUs (e.g., L20 at 119.5 dense FP16 TFLOPS), this IO-for-precision trade-off is strictly preferable to recomputing S and dP across D-chunks, which would consume precious tensor core throughput. On SM≥90 GPUs with abundant compute, the recompute approach (§3.3) becomes viable.

3.3 Hopper-Specialized Algorithm (CuTe DSL, SM≥90, $D = 512$)

On Hopper GPUs, the WGMMA (Warp Group Matrix Multiply-Accumulate) instruction can process a full 512×64 tile in a single operation. This enables a qualitatively different algorithm design that is *not* a straightforward port of the generic Split-D kernel.

3.3.1 Forward: Full-D $QK^\top$

The CuTe DSL forward pass performs $QK^\top$ as a **single full-D WGMMA**—no D-chunk loop. The Q tile of shape $[B_r, 512]$ and K tile of $[512, B_c]$ are loaded once via TMA (Tensor Memory Accelerator) and multiplied in one instruction. The softmax and PV phases are then split along the *output column* dimension (Split-N) rather than the head dimension:

- **WG1** (Consumer-QK): performs the full-D WGMMA, computes online softmax, and produces the PV-front half (gO columns 0:256) plus LSE.
- **WG2** (Consumer-PV): consumes the softmax output from WG1 and computes the PV-back half (gO columns 256:512).

WG0 (warps 0–3, Producer) issues TMA loads for Q, K, and V in a double-buffered fashion, overlapping data movement with computation.

Algorithm 1 Generic Split-D Forward Pass (single program)

Input: $Q, K, V \in \mathbb{R}^{N \times D}$, scale $\alpha = 1/\sqrt{D}$
Output: $O \in \mathbb{R}^{N \times D}$, LSE $\in \mathbb{R}^{N}$
Hyperparams: $B_r, B_c, D_{\text{qk}}, D_v$, $N_{\text{qk}} = \lceil D/D_{\text{qk}} \rceil$, $N_v = \lceil D/D_v \rceil$

$i \leftarrow \text{program\_id}()$ ▷ owns Q-block i
$\text{offs\_m} \leftarrow [iB_r, \ldots, (i+1)B_r - 1]$
$m_i \leftarrow (-\infty)^{B_r}$; $\ell_i \leftarrow \mathbf{0}^{B_r}$
for $v = 0$ **to** $N_v - 1$ **do**
    $O_i^{(v)} \leftarrow \mathbf{0} \in \mathbb{R}^{B_r \times D_v}$
for $j = 0$ **to** $\lceil N/B_c \rceil - 1$ **do**
    $\text{offs\_kv} \leftarrow [jB_c, \ldots, (j+1)B_c - 1]$
    // **Phase 1: Split-D QK accumulation**
    $S \leftarrow \mathbf{0} \in \mathbb{R}^{B_r \times B_c}$
    for $c = 0$ **to** $N_{\text{qk}} - 1$ **do**
        $\text{offs\_d} \leftarrow [cD_{\text{qk}}, \ldots, (c+1)D_{\text{qk}} - 1]$
        $q \leftarrow \text{load}(Q[\text{offs\_m}, \text{offs\_d}])$
        $k \leftarrow \text{load}(K[\text{offs\_kv}, \text{offs\_d}])$
        $S \leftarrow S + q\,k^\top$
    $S \leftarrow \alpha \cdot S$; apply mask/bias/causal
    // **Phase 2: Online softmax**
    $m_{\text{new}} \leftarrow \max(m_i, \text{rowmax}(S))$
    $\alpha_{\text{rescale}} \leftarrow \exp(m_i - m_{\text{new}})$
    $P \leftarrow \exp(S - m_{\text{new}})$; apply dropout
    $\ell_i \leftarrow \ell_i \cdot \alpha_{\text{rescale}} + \text{rowsum}(P)$
    // **Phase 3: Split-D PV accumulation**
    for $v = 0$ **to** $N_v - 1$ **do**
        $\text{offs\_d} \leftarrow [vD_v, \ldots, (v+1)D_v - 1]$
        $v_{\text{tile}} \leftarrow \text{load}(V[\text{offs\_kv}, \text{offs\_d}])$
        $O_i^{(v)} \leftarrow \alpha_{\text{rescale}} \odot O_i^{(v)} + P\,v_{\text{tile}}$
    $m_i \leftarrow m_{\text{new}}$
// **Phase 4: Final normalization and writeback**
for $v = 0$ **to** $N_v - 1$ **do**
    $O_i^{(v)} \leftarrow O_i^{(v)} \oslash (\ell_i \oplus \varepsilon)$
    $\text{store}(O[\text{offs\_m}, \text{offs\_d}], O_i^{(v)})$
$\text{store}(\text{LSE}[\text{offs\_m}], m_i + \log \ell_i)$

3.3.2 Backward: Split-D with S,dP Recompute

The backward pass returns to Split-D ($D_{\text{chunk}} = 256$, 2 passes), but leverages Hopper's compute headroom to **recompute** S and dP across D-chunks, eliminating the bf16 round-trip that the generic algorithm must tolerate or pay IO to avoid.

The dKdV kernel uses a 2-warpgroup design with explicit named barriers:

- **WG1** (warps 4–7): recomputes $S_{i(j)} = Q_i K_j^\top$ via 2 WGMMA sub-steps per D-chunk, applies mask and softmax, then writes P to shared memory in **both** bf16 (for WG1's own dV GEMM) and fp32 (for WG2's dS computation).
- **WG2** (warps 8–11): recomputes $dP = dO\,V^\top$ via 2 WGMMA sub-steps, then reads P from the **fp32 channel** to compute $dS = P \odot (dP - \Delta)$ without any bf16 round-trip.

WG0 (warps 0–3, Producer) issues TMA loads for K and V, persisting them across both D-passes to amortize HBM traffic.

Why recompute is viable here. The additional GEMM operations to re-materialize S and dP consume approximately 40% more WGMMA throughput compared to a non-recompute design. On Hopper GPUs with ∼989 dense FP16 TFLOPS of tensor core throughput, this overhead is absorbed by the abundant compute

Algorithm 2 Generic Split-D Backward Pass — dKdV and dQ (single program)

Input: $Q, dO, O \in \mathbb{R}^{N \times D}$, saved LSE $\in \mathbb{R}^N$
Output: dK, dV (dKdV kernel) or dQ (dQ kernel)
Hyperparams: $B_r, B_c, D_{\text{qk}}, D_v$

dKdV Kernel (owns K-block j):
  Load K-block K_j, V-block V_j into SMEM (persisted across Q-blocks)
for $i = 0$ **to** $\lceil N/B_r \rceil - 1$ **do**
    // **Phase 1: Recompute S, P via Split-D QK**
    $S \leftarrow \mathbf{0}$; accumulate $q\,k^\top$ over D_{qk}-chunks
    Compute $P_{i(j)} = \exp(\alpha S - \text{LSE}_i)$
    // **Phase 2: dV via Split-D PV**
  for $v = 0$ **to** $N_v - 1$ **do**
      $dV_j^{(v)} \leftarrow dV_j^{(v)} + P_{i(j)}^\top dO_{\text{chunk}}$
    // **Phase 3: dS and dK**
    $dP_{i(j)} \leftarrow \sum_v dO_{\text{chunk}} V_{\text{chunk}}^\top$ (Split-D dP)
    $dS_{i(j)} \leftarrow P_{i(j)} \odot (dP_{i(j)} - \Delta_i)$
  for $c = 0$ **to** $N_{\text{qk}} - 1$ **do**
      $dK_j^{(c)} \leftarrow dK_j^{(c)} + dS_{i(j)}^\top q$ (Split-D dK)

dQ Kernel (owns Q-block i, symmetric to dKdV):
  The outer loop iterates over K-blocks; inner loop recomputes $S_{i(j)}$ and $dS_{i(j)}$ as above, then accumulates $dQ_i \leftarrow dQ_i + dS_{i(j)} K_j$ via Split-D D_{qk}-chunks.
  A pre-processing kernel computes $\Delta_i = \text{rowsum}(dO_i \odot O_i)$, available to both dKdV and dQ.

headroom, while the fp32 precision channel guarantees that the softmax gradient—the most numerically sensitive operation in attention backward—suffers no bf16 degradation.

3.4 Backward Precision Strategy: A Hardware-Driven Trade-off

The backward precision challenge is specific to the backward pass. The forward pass has no such issue: Split-D forward uses online softmax with fp32 accumulation for m_i and ℓ_i, keeping output error within standard mixed-precision bounds ($\leq 6 \times 10^{-3}$ in BF16) regardless of storage dtype. Consequently, for inference-only deployments (forward pass only), FFPA works correctly on any GPU without any precision-protection mechanism.

For training (forward + backward), Table 1 summarizes the two backward precision strategies and the hardware characteristics that motivate each choice. On compute-limited SM≤89 GPUs, we pay an IO cost; on compute-rich SM≥90 GPUs, we pay a compute cost. The common goal is eliminating bf16 round-trip errors in gradient accumulation.

Table 1: Backward pass precision strategies. The forward pass has no precision issue and works correctly on any GPU.

	SM≤89 (L20)	**SM≥90 (H200)**
Strategy	fp32 buffer	recompute + fp32 channel
Extra cost	IO (2× bw)	Compute (extra GEMM)
Feasibility	IO ≪ recompute	Surplus compute absorbs
Precision	fp32 accum., no bf16 loss	fp32 P, no bf16 loss

3.5 Multi-Backend Dispatch

FFPA exposes a unified `ffpa_attn_func` API that selects the appropriate backend at runtime:

- $D \leq 256$: fallback to PyTorch SDPA (cuDNN FlashAttention).

- $D > 256$, SM≥90, $D \in [320, 512]$: CuTe DSL backend (Hopper-specialized).
- $D > 256$, SM≥80: Triton backend (default, with optional warp-specialized TMA mode on SM≥90).
- CUDA backend available as a forward-only reference implementation.

Triton kernels support persistent autotuning that pre-computes optimal block sizes $(B_r, B_c, D_{\text{qk}}, D_v)$ and pipeline stages per GPU model, stored as JSON configurations loaded at import time.

4 Experiments

We evaluate FFPA across many GPU architectures, organized by compute capability: Hopper (H200, H800, H20), Blackwell (RTX 5090), and Ada (L20).

4.1 Experimental Setup

Hardware. NVIDIA H200 SXM (Hopper, 989 dense FP16 TFLOPS), NVIDIA H800 PCIe (Hopper, 756 dense FP16 TFLOPS), NVIDIA GeForce RTX 5090 (Blackwell, 419 dense FP16 TFLOPS), NVIDIA H20 (Hopper, 148 dense FP16 TFLOPS), NVIDIA L20 (Ada Lovelace, 119.5 dense FP16 TFLOPS). All experiments use PyTorch 2.11 and CUDA 13.0.

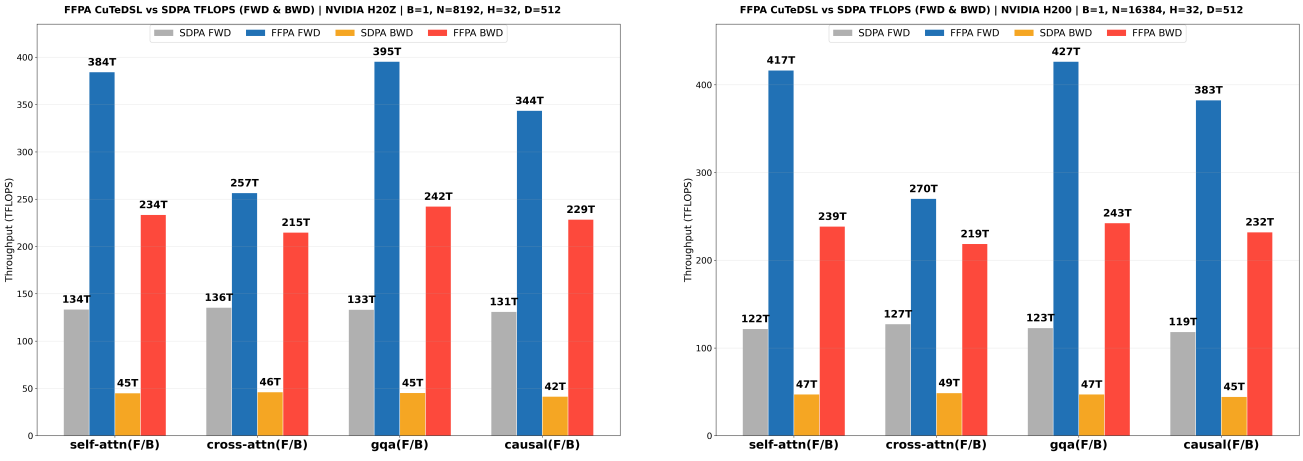
Baseline. We compare against PyTorch's `F.scaled_dot_product_attention` (SDPA) with the "efficient" backend. For $D > 256$, cuDNN FlashAttention is not supported, so SDPA falls back to a memory-efficient Triton kernel in all our measurements.

Metrics. Wall-clock latency (milliseconds, median of 10 iterations after 2 warmup), effective TFLOPS computed from the theoretical attention FLOP count ($4N^2D$ forward, $10N^2D$ backward for self-attention), and speedup relative to SDPA. Numerical error is verified via $\max|O_{\text{FFPA}} - O_{\text{SDPA}}|$.

Default configuration. $B = 1$, $H = 32$, $N = 8192$, $D = 512$, BF16 compute with FP32 accumulation. FFPA Triton backend uses "max" autotune mode.

4.2 Hopper Results (H200, H800, H20)

Table 2 reports self-attention results on three Hopper GPUs. On H200 with CuTe DSL, forward reaches **372 TFLOPS** (2.75×) at N=8192 and **416 TFLOPS** (3.39×) at N=16384; backward achieves 4.94×–4.99×. GQA peaks at **426 TFLOPS** (Table 5). H800 Triton delivers 203 TFLOPS forward (2.64×) and 87 TFLOPS backward (2.53×) at 8K. H20 Triton achieves 112 TFLOPS forward (1.77×) and 92 TFLOPS backward (3.26×), highlighting Split-D's effectiveness on compute-limited hardware.



(a) N=8192 (b) N=16384

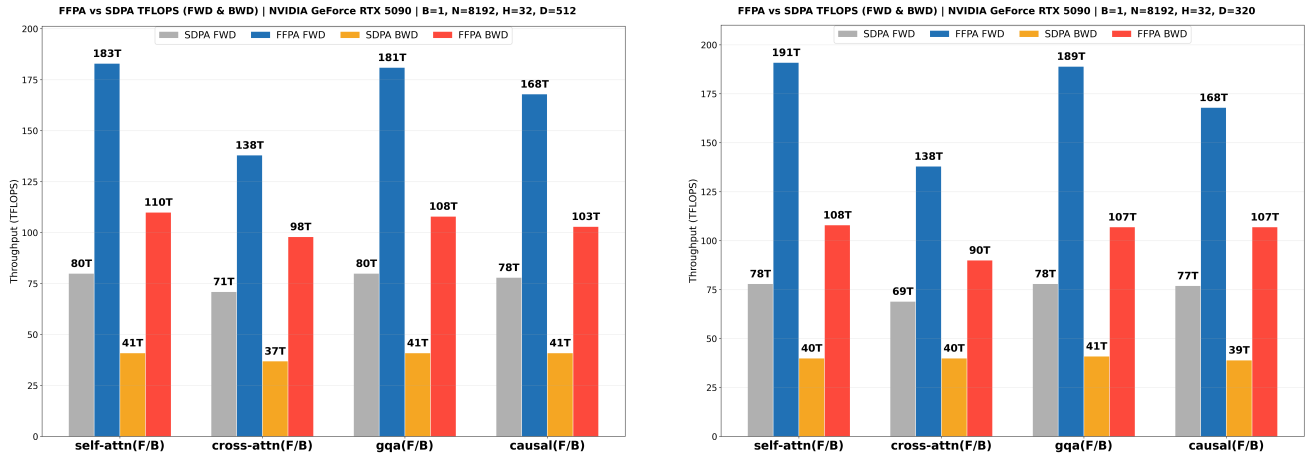
Figure 2: H200 CuTe DSL TFLOPS comparison vs. SDPA (D=512).

Table 2: Hopper GPU self-attention results (B=1, H=32, D=512, BF16). TFLOPS as FFPA / SDPA; counts attention GEMM FLOPs only. H800 latency derived from reported TFLOPS.

GPU (Backend)	N	Forward			Backward		
		Latency (ms)	TFLOPS	Speedup	Latency (ms)	TFLOPS	Speedup
H200 CuTeDSL	8192	11.8	372 / 135	**2.75**	48.4	227 / 46	**4.94**
	16384	42.3	416 / 123	**3.39**	187.0	235 / 47	**4.99**
H800 Triton	8192	21.7	203 / 77	**2.64**	126.3	87 / 34	**2.53**
H20 Triton	8192	39.1	112 / 64	**1.77**	119.3	92 / 28	**3.26**

4.3 Blackwell Results (RTX 5090)

Figure 3 shows FFPA Triton results on the RTX 5090 (Blackwell, SM120, 419 dense FP16 TFLOPS). At D=512, FFPA achieves **2.08**× forward and **2.49**× backward speedup over SDPA. The Split-D chunk size $D_{\text{qk}} = 64$ is independent of D, so register pressure per chunk remains constant while FLOPs scale linearly, enabling the same kernel to serve head dimensions up to $D = 1024$—a regime where standard FlashAttention kernels are entirely infeasible—without re-tuning. Figure 3 shows the TFLOPS comparison at D=512 and D=320; additional plots for $D \in \{640, 1024\}$ confirm consistent gains across all head dimensions.



(a) D=512 (b) D=320

Figure 3: RTX 5090 Triton TFLOPS comparison vs. SDPA.

4.4 Ada Results (L20)

Table 3 shows L20 results using the generic Triton backend (no TMA, no warp specialization). On L20 (Ada Lovelace, 119.5 dense FP16 TFLOPS peak), FFPA achieves 1.65× forward speedup at $D = 512$. The backward pass supports two precision modes: a **bf16 gradient buffer** (default, memory-efficient) at 1.84×, and an **fp32 gradient buffer** (precision-safe, higher IO) at 1.49×. On compute-limited SM≤89 GPUs, the bf16 buffer is the default choice; the fp32 buffer is available when maximum numerical precision is required.

Table 3: L20 Triton, self-attention latency in ms (B=1, H=32, N=8192, BF16). Bwd bf16 buffer: FFPA 192.1 ms vs. SDPA 353.2 ms; fp32 buffer: FFPA 250.0 ms vs. SDPA 373.6 ms.

D	Forward			Backward			
	FFPA (ms)	SDPA (ms)	Speedup	FFPA (ms)	SDPA (ms)	Speedup (bf16)	Speedup (fp32)
512	45.4	74.8	**1.65**	192.1	353.2	**1.83**	**1.49**

4.5 Attention Variants

FFPA supports the full range of attention variants: causal masking, GQA/MQA, cross-attention, decode attention ($N_q = 1$), additive bias, and dropout. Table 4 reports speedups across three GPUs (H200 CuTe DSL, H20 Triton, L20 Triton) at $D = 512$, $N = 8192$. Speedups are consistent across all variants (1.34–6.05×), with the largest gain on decode attention backward where SDPA's per-token kernel launch overhead is most severe.

Table 4: Attention variant **speedups** (B=1, H=32, N=8192, D=512, BF16).

Variant	H200 CuTeDSL		H20 Triton		L20 Triton		
	Fwd	Bwd	Fwd	Bwd	Fwd	Bwd (bf16)	Bwd (fp32)
Self-attn	2.75	4.94	1.77	3.26	1.65	1.83	1.49
Cross (N_q=1024)	1.81	4.15	1.57	3.34	1.59	1.63	1.21
Decode (N_q=1)	—	—	3.31	6.05	1.03	2.20	2.20†
GQA (H_{kv}=8)	2.98	5.18	1.74	3.26	1.65	1.79	1.47
Causal	2.64	5.15	1.81	3.15	1.49	2.00	1.55
Attn mask	—	—	1.62	2.80	1.56	1.93	1.44
Dropout (p=0.1)	—	—	1.59	3.03	1.55	1.74	1.34
Non-aligned	2.59	6.00	1.76	3.40	1.60	1.91	1.54

† Decode-attn uses fp32 GEMV; fp32 buffer has no effect.

Table 5: H200 CuTeDSL variant results (B=1, H=32, BF16). TFLOPS as FFPA / SDPA.

Variant	Forward			Backward		
	TFLOPS	SDPA	Speedup	TFLOPS	SDPA	Speedup
D=512, N=8192						
Self-attn	372	135	**2.75**	227	46	**4.94**
Cross (N_q=1024)	247	137	**1.81**	193	47	**4.15**
GQA (H_{kv}=8)	403	135	**2.98**	238	46	**5.18**
Causal	347	131	**2.64**	217	42	**5.15**
Non-aligned	359	139	**2.59**	233	39	**6.05**
D=512, N=16384						
Self-attn	416	123	**3.39**	235	47	**4.99**
Cross (N_q=1024)	264	127	**2.08**	198	48	**4.10**
GQA (H_{kv}=8)	426	123	**3.46**	240	47	**5.10**
Causal	381	119	**3.21**	226	44	**5.12**
Non-aligned	403	131	**3.09**	240	43	**5.53**
D=320, N=8192						
Self-attn	305	138	**2.21**	164	43	**3.82**
Cross (N_q=1024)	216	141	**1.54**	154	44	**3.51**
GQA (H_{kv}=8)	332	140	**2.37**	162	43	**3.77**
Causal	267	135	**1.98**	151	40	**3.77**
Non-aligned	297	143	**2.08**	168	37	**4.57**
D=320, N=16384						
Self-attn	322	128	**2.52**	161	45	**3.60**
Cross (N_q=1024)	221	133	**1.66**	154	46	**3.35**
GQA (H_{kv}=8)	335	128	**2.61**	172	45	**3.84**
Causal	297	129	**2.30**	151	43	**3.55**
Non-aligned	314	129	**2.44**	170	42	**4.03**

Table 5 extends the evaluation to $N = 16384$ and $D = 320$ on H200 CuTe DSL, covering four configuration groups. At $D = 512$ and $N = 8192$, backward speedups reach 6.05× (non-aligned); at $N = 16384$, forward

throughput scales to 426 TFLOPS (GQA, 3.46×). At the moderate head dimension $D = 320$, speedups remain substantial: 2.08–4.57× at $N = 8192$ and 1.66–4.03× at $N = 16384$, confirming that Split-D is effective across a range of head dimensions and sequence lengths, not only at $D = 512$.

4.6 End-to-End Training

We validate FFPA in a realistic training setting on Gemma4-31B [7], an open-weights multimodal language model whose attention layers are split into two groups: 50 sliding-window layers with $D = 256$ (which remain on SDPA) and 10 full-attention layers with $D = 512$ (accelerated by FFPA). The experiment runs on a single node of 8×H200 GPUs with FSDP2 (data-parallel degree 8, global batch size 8, gradient accumulation 4) and activation checkpointing at sequence length 8K.

At the **same memory footprint** as the SDPA baseline (∼65 GiB per GPU), FFPA CuTe DSL delivers **1.4–1.5**× higher end-to-end training throughput. The training loss curve overlaps the SDPA baseline within normal bf16 noise, confirming that the Split-D forward and the recompute-based backward preserve full numerical fidelity in a production training loop.

This is a striking result: only 10 out of 60 attention layers (17%) use $D = 512$ and are accelerated by FFPA, yet the overall throughput improves by 1.4–1.5×—the remaining 50 layers run unmodified SDPA. It underscores both the disproportionate impact of large-D attention on end-to-end training cost and the effectiveness of Split-D in mitigating it. With FFPA, the previously dominant $D = 512$ layers become performance-neutral, shifting the optimization focus to other components of the model pipeline.

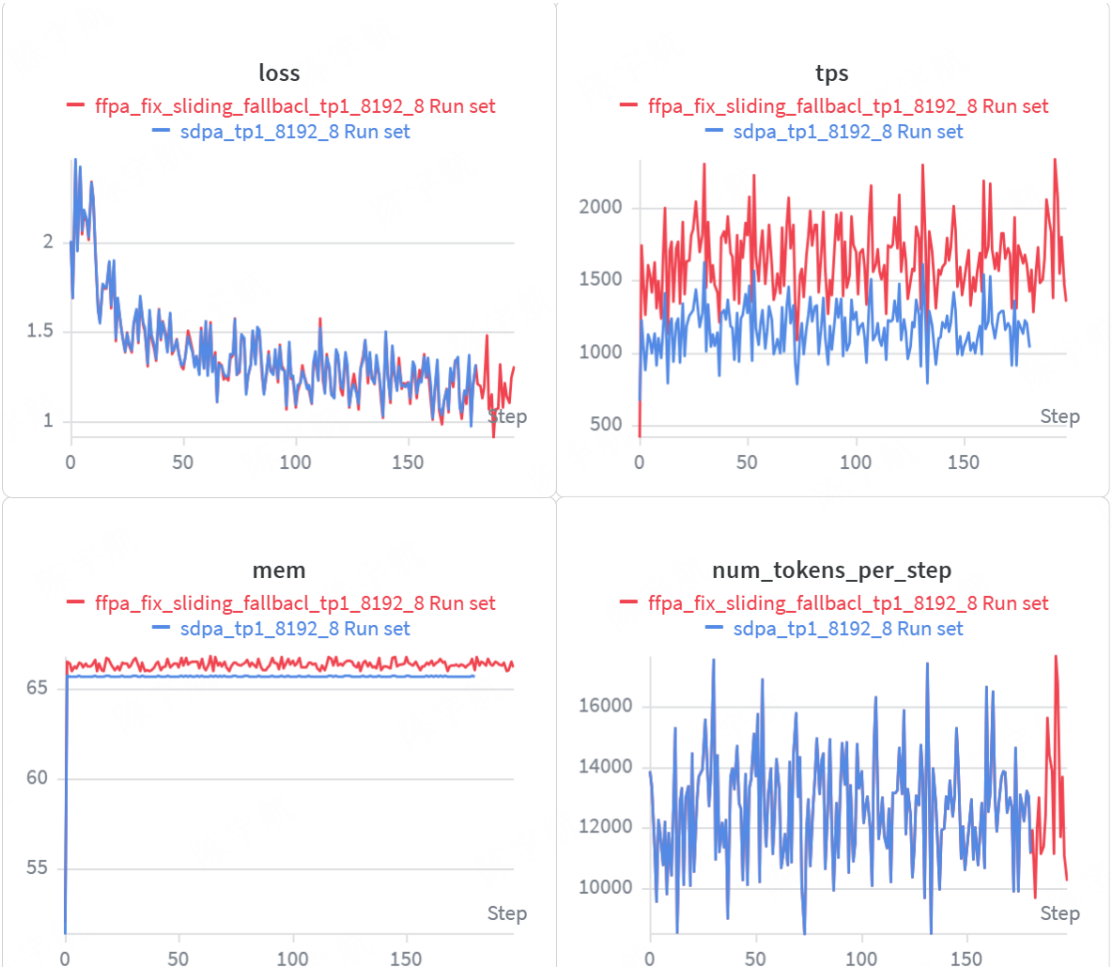


Figure 4: End-to-end training throughput on Gemma4-31B (8×H200, FSDP2, 8K sequence length). FFPA accelerates the 10 D=512 attention layers, achieving 1.4–1.5× overall throughput improvement at the same memory footprint as SDPA.

4.7 Ablation Study

We analyze the contribution of key design choices using the Triton backend on L20 (N=8192, D=512).

Split-D chunk size. Increasing D_{qk} from 64 to 128 reduces the number of QK accumulation steps by half but increases register pressure per chunk. Autotuning selects $D_{\text{qk}} = 64$ for $D = 512$ on L20, indicating that the reduced register pressure outweighs the benefit of fewer loop iterations. On H800 with larger register files (256 KB per SM), autotuning occasionally selects $D_{\text{qk}} = 128$.

Warp specialization and TMA. On SM≥90 GPUs, enabling TMA with warp specialization yields **44%–69%** higher throughput over the generic Triton baseline (Table 6 shows H200 Triton without TMA/WS; compare with CuTe DSL results in Table 2). The gain comes from two sources: (1) TMA offloads address computation and data movement from the compute warps, and (2) warp specialization enables concurrent data movement and GEMM execution through hardware warp schedulers. On SM≤89 GPUs without TMA support, these features are unavailable, making the fp32 buffer the primary precision-protection mechanism.

Table 6: H200 Triton baseline without TMA or warp specialization (B=1, H=32, N=8192, D=512, BF16). TFLOPS as FFPA / SDPA.

Variant	Forward			Backward		
	TFLOPS	SDPA	Speedup	TFLOPS	SDPA	Speedup
Self-attn	259	136	**1.91**	134	46	**2.93**
Cross (N_q=1024)	232	138	**1.69**	106	47	**2.29**
Decode (N_q=1)	2.1	0.70	**2.96**	1.7	0.38	**4.48**
GQA (H_{kv}=8)	261	136	**1.93**	133	46	**2.92**
Causal	284	132	**2.16**	109	42	**2.59**
Attn mask	207	130	**1.59**	94	45	**2.12**
Dropout (p=0.1)	188	127	**1.48**	105	45	**2.32**
Non-aligned	245	141	**1.73**	127	39	**3.29**

4.8 Numerical Accuracy

Table 7 summarizes the numerical accuracy of FFPA relative to SDPA across compute precisions and hardware configurations. The forward pass is well-behaved: FP16 achieves $\max|O_{\text{FFPA}} - O_{\text{SDPA}}| \leq 5 \times 10^{-4}$, while BF16 reaches $\leq 6 \times 10^{-3}$, both within expected bounds for mixed-precision attention (FP16 10-bit mantissa vs. BF16 7-bit).

The backward pass is more sensitive. With the default bf16 gradient buffer on SM≤89 GPUs, the load-modify-store pattern introduces bf16 round-trip errors, and the worst-case discrepancy reaches $\max|dV| \approx 8 \times 10^{-2}$ in BF16 causal backward. Enabling the **fp32 gradient buffer** on these GPUs reduces the error to $\leq 2 \times 10^{-2}$ at the cost of 2× gradient IO bandwidth. On SM≥90 GPUs with the CuTe DSL backend, the fp32 precision channel in the recompute-based backward achieves the same $\leq 2 \times 10^{-2}$ bound without the IO penalty, by re-materializing the softmax and its gradient in fp32 within the kernel.

Table 7: Numerical accuracy vs. SDPA (B=1, H=32, N=8192, D=512). Lower is better.

	FP16 Fwd	BF16 Fwd	BF16 Bwd (causal, $\max\|dV\|$)
SM≤89, bf16 buffer	$\leq 5 \times 10^{-4}$	$\leq 6 \times 10^{-3}$	$\approx 8 \times 10^{-2}$
SM≤89, fp32 buffer	—	—	$\leq 2 \times 10^{-2}$
SM≥90, CuTe DSL	$\leq 5 \times 10^{-4}$	$\leq 6 \times 10^{-3}$	$\leq 2 \times 10^{-2}$

5 Conclusion

Split-D is an effective and general solution to the large head-dimension bottleneck. We presented FFPA, an attention algorithm that extends FlashAttention to $D > 256$ via Split-D, reducing SRAM complexity from $O(B_c \times D)$ to $O(1)$ and keeping active register pressure bounded at $O(D_{\text{chunk}})$. FFPA provides a hardware-driven precision strategy: fp32 gradient buffers for compute-limited SM≤89 GPUs, and recompute-based precision

for compute-rich SM≥90 GPUs with the CuTe DSL backend, which further exploits full-D WGMMA in the forward pass. Across many GPU architectures, FFPA achieves 1.5×–6.1× micro-benchmark speedup and reaches 426 TFLOPS on H200. On Gemma4-31B with 8×H200 GPUs, accelerating only the 10 D=512 layers delivers 1.4–1.5× end-to-end training throughput improvement (§4).

Split-D is a general algorithmic primitive for large inner-dimension kernels. The head-dimension chunking strategy is not specific to attention: any operator whose inner dimension exceeds the practical limits of shared memory and register files can benefit from recursive tiling along that dimension. This work demonstrates its effectiveness across CUDA, Triton, and CuTe DSL backends spanning five GPU architectures, and establishes Split-D as a reusable pattern for the growing class of large-D models in diffusion, video generation, and multimodal learning.

Split-D extends naturally to future hardware and workloads. Future work includes FP8 computation for Hopper and Blackwell GPUs, integration with sequence-parallel attention (Ulysses CP/TP) for distributed inference, and application of Split-D to other large-inner-dimension operations beyond attention.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *International Conference on Learning Representations (ICLR)*, 2024.

[4] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv:2407.08608*, 2024.

[5] W. Peebles and S. Xie, "Scalable diffusion models with transformers," *International Conference on Computer Vision (ICCV)*, 2023.

[6] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh, "Video generation models as world simulators," 2024.

[7] Google DeepMind, "Gemma 4: Byte for byte, the most capable open models," `https://huggingface.co/google/gemma-4-31B`, 2026.

[8] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.

[9] NVIDIA Corporation, "CUTLASS: CUDA templates for linear algebra subroutines," `https://github.com/NVIDIA/cutlass`, 2024.

[10] NVIDIA Corporation, "CuTe: CUDA Templates," in CUTLASS 3.x, `https://github.com/NVIDIA/cutlass`, 2024.